Project Title

SPIN SHOT

Project Overview:

An arcade-like shooter game that tests the user's visual accuracy and reaction time. Align the barrel and click the trigger to launch a speedy arrow, hoping to pass through the targets/circles while avoiding the deadly spinning wheel.

1. Electronics Details:

Components Used: DC motor, Servo motor, Push buttons, Power module (initial plan: to use IR sensor and buzzer)

2. Mechanical Details:

The Process: After finalising the idea, we brainstormed on the functioning components we would require, their placements, and how they would look.

Starting with the target box, we ideated on multiple ways to incorporate an obstacle/circular wheel. Its size corresponds to the size of the box on which it rotates. We then looked for simple shooting mechanisms using different materials — first, a 3D print using filaments, and second, using MDF.

To have a moving turret in both the horizontal and vertical planes, we explored different forms until we reached an efficient structure that consumed fewer materials and functioned smoothly with minimal error.

The files for laser-cutting were done on LibreCAD, which excels in 2D design, enhancing accuracy in measurement and finish.

The 3D shooting mechanism was inspired by a YouTube video on "3D printed shooter ideas," which also helped us understand how we could build the same using MDF.

3. Software Details: On MIT App Inventor, we structured and designed the application through which users first scan for nearby devices, connect, and then provide inputs — our input options are "1", "2", … "6".

We then use Thonny, where the code responds to the app, receives the inputs, processes them, and provides output through our selected components. The three servo motors are coded for step-by-step movement, controlled individually by the user. The DC motor is coded to have a modified RPM as per our requirements.

The Code:

```
“from machine import PWM, Pin

import bluetooth

import time
```

```
# 1. Add position variables at the top
pos1 = 77  # Starting middle position for mot1 (approx 90 degrees)
pos2 = 77  # Starting middle position for mot2
step = 1   # How many duty units to move per click

mot1=PWM(Pin(33))
mot1.freq(50)

mot2=PWM(Pin(32))
mot2.freq(50)

motrig=PWM(Pin(15))
motrig.freq(50)
value = ""
name = "Shooter36" #Name of Your ESP32 (Change it to avoid Confusion)
ble = bluetooth.BLE()

ble.active(False)
time.sleep(0.5)
ble.active(True)
ble.config(gap_name=name)

service_UUID = bluetooth.UUID("6e400001-b5a3-f393-e0a9-e50e24dcca9e")
char_UUID = bluetooth.UUID("6e400002-b5a3-f393-e0a9-e50e24dcca9e")

char = (char_UUID, bluetooth.FLAG_WRITE)
service = (service_UUID, (char,),)
((char_handle,),) = ble.gatts_register_services((service,))

def event_occured(event, data):
```

```
    if event == 1:
        print("Connected")

    elif event == 2:
        print("Disconnected")
        advertise(name)

    elif event == 3:
        conn_handle, value_handle = data
        if value_handle == char_handle:
        #reading the Value written on characteristics by Phone/client
            raw_msg = ble.gatts_read(char_handle).rstrip(b'\x00')
            msg = raw_msg.decode().strip()
            print("Received:", msg)

            global value
            value = msg

def advertise(device_name):

    name_bytes = device_name.encode()

    flags = bytearray([0x02, 0x01, 0x06])
    short_name = bytearray([len(name_bytes) + 1, 0x08]) + name_bytes
    full_name = bytearray([len(name_bytes) + 1, 0x09]) + name_bytes
    adv_data = flags + short_name + full_name

    ble.gap_advertise(50, adv_data=adv_data)
    print("Awating Connection...Advertising as:", device_name)
```

```
advertise(name)

ble.irq(event_occured)

while True:
    if value == "1":
        if pos1 < 125: # Upper limit safety
            pos1 += step
            mot1.duty(pos1)
            print("Mot1 Right:", pos1)
        value = "" # CRITICAL: Reset value so it doesn't loop continuously

    elif value == "2":
        if pos1 > 25: # Lower limit safety
            pos1 -= step
            mot1.duty(pos1)
            print("Mot1 Left:", pos1)
        value = ""

    elif value == "3":
        if pos2 < 125:
            pos2 += step
            mot2.duty(pos2)
            print("Mot2 Up:", pos2)
        value = ""

    elif value == "4":
        if pos2 > 25:
            pos2 -= step
            mot2.duty(pos2)
            print("Mot2 Down:", pos2)
```

```
        value = ""

    elif value == "5":
        motrig.duty(45)
        print("Trigger On")
        value = ""

    elif value == "6":
        motrig.duty(77)
        print("Trigger Off")
        value = ""
    time.sleep(0.01)”
```

The Function: Two of the three servo motors are coded to move one step at a time, placing the responsibility of aligning the turret on the user. The third servo motor works similarly, releasing the rubber band from the crossbow to launch the arrow (MDF). The target box has three holes and shields the mechanism inside. It has a circular wheel in front of the holes, which rotates at a modifiable speed controlled using push buttons. Each button is connected to a different duty cycle, creating three different speeds corresponding to three difficulty levels (Easy to Hard).

4. Project Documents: We aim to make a fun game that engages users and tests their reactive skills across different rotational speed levels displayed by the obstacle/circular wheel.

5. Media: A video with an end-to-end detailed walkthrough of our project and user testing.

⚙️ How to Navigate This Repository

Electronics Details → Circuit, components justification

Mechanical Details → Sketches, CAD, and fabrication

Software Details → Code, logic, and implementation

Project Documents → Design thinking, planning, testing

Team Members:

Radhika Madhavi

Kedar Nayak